

Design Patterns

Clément RENIERS

Table des matières

Stratégie	2
Principe	2
Exemple (Tri)	2
État	3
Principe	3
Exemple (Lecteur audio)	3
Observateur	4
Principe	4
Exemple (Chaîne YouTube)	4
Singleton	5
Principe	5
Exemple	5
Décorateur	6
Principe	6
Exemple	6
Usine (Factory)	7
Principe	7
Exemple (Véhicules)	7
Visiteur	8
Principe	8
Exemple (Debug d'une figure géométrique)	8
Composite	9
Principe	9
Exemple (Interface graphique)	9
Itérateur	10
Principe	10
Exemple (Bibliothèque)	10
Façade	11
Template	12

Stratégie

Principe

⇒ On dépend des classes abstraites, rajouter une stratégie ne change rien au code.

⇒ Utilisable quand on a plusieurs variantes d'un algorithme.

⇒ Au contraire du pattern **État**, on change de stratégie de manière explicite. Les stratégies ne sont pas liées à un état de l'application.

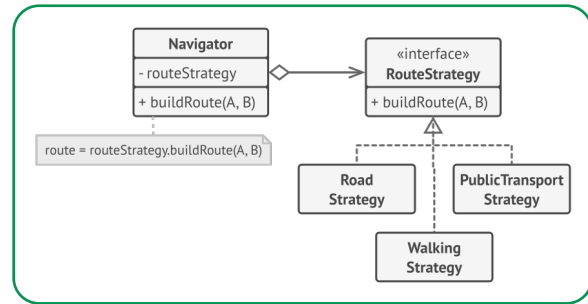


Figure 1: Utilisation simple du pattern Stratégie

Exemple (Tri)

```
1 public interface SortingStrategy {
2     void sort(int[] array);
3 }
4
5 public class SortingContext {
6     private SortingStrategy sortingStrategy;
7
8     public SortingContext(SortingStrategy sortingStrategy) {
9         this.sortingStrategy = sortingStrategy;
10    }
11
12    public void setSortingStrategy(SortingStrategy sortingStrategy) {
13        this.sortingStrategy = sortingStrategy;
14    }
15
16    public void performSort(int[] array) {
17        sortingStrategy.sort(array);
18    }
19 }
```

```
1 public interface SortingStrategy {
2     void sort(int[] array);
3     // Implement sorting logic here
4 }
```

```
1 // BubbleSortStrategy
2 public class BubbleSortStrategy implements SortingStrategy {
3     @Override
4     public void sort(int[] array) {
5         // Implement Bubble Sort algorithm
6         System.out.println("Sorting using Bubble Sort");
7     }
8 }
9
10 // MergeSortStrategy
11 public class MergeSortStrategy implements SortingStrategy {
12     @Override
13     public void sort(int[] array) {
14         // Implement Merge Sort algorithm
15         System.out.println("Sorting using Merge Sort");
16     }
17 }
18
19 // QuickSortStrategy
20 public class QuickSortStrategy implements SortingStrategy {
21     @Override
22     public void sort(int[] array) {
23         // Implement Quick Sort algorithm
24         System.out.println("Sorting using Quick Sort");
25     }
26 }
```

État

Principe

- ⇒ Sert dans le cas d'une classe qui doit avoir un comportement différent selon les états dans lesquels elle se trouve.
- ⇒ Permet d'éviter les gros blocs conditionnels
- ⇒ C'est l'état qui se charge de réaliser les actions.
- ⇒ Différence avec **Stratégie** : le comportement peut-être changé dans la durée de vie de la classe

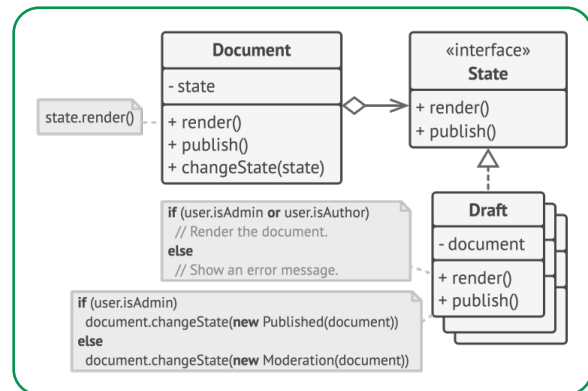


Figure 2: Utilisation simple du pattern État

Exemple (Lecteur audio)

- ⇒ On imagine un lecteur audio avec un unique bouton pour play/pause.
- ⇒ On modélise deux états : “Playing” et “Paused”. Le comportement du bouton change selon l'état dans lequel on se trouve.

```
1 public class Reader {
2     private ReaderState _state = null;
3
4     public Reader(ReaderState state) {
5         this._state = state;
6     }
7
8     public void PressPlay() {
9         this._state.PressPlay(this);
10    }
11
12    public ReaderState CurrentState {
13        get { return _state; }
14        set { _state = value; }
15    }
16 }
17
18 public abstract class ReaderState {
19     public abstract void PressPlay(Reader reader);
20 }
21
22 public class ReaderPlayingState extends ReaderState {
23     public ReaderPlayingState() {
24         System.out.println("Reader playing");
25     }
26
27     @Override
28     public void PressPlay(Reader reader) {
29         reader.CurrentState = new ReaderPausedState();
30     }
31 }
32
33 public class ReaderPausedState extends ReaderState {
34     public ReaderPausedState() {
35         System.out.println("Reader paused");
36     }
37
38     @Override
39     public void PressPlay(Reader reader) {
40         reader.CurrentState = new ReaderPlayingState();
41     }
42 }
```

Observateur

Principe

⇒ Permet à un sujet de notifier ses observateurs lorsqu'il change d'état. Les observateurs peuvent alors réagir à ce changement.

⇒ Le sujet ne connaît pas les observateurs, il leur envoie juste une notification.

⇒ Utilisé dans les interfaces graphiques, les systèmes d'événements, etc.

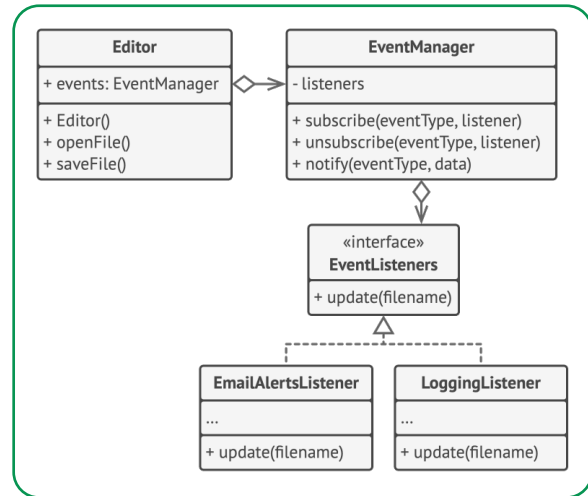


Figure 3: Utilisation simple du pattern Observateur

Exemple (Chaîne YouTube)

⇒ On modélise une chaîne YouTube suivie par des abonnés. Lorsqu'une nouvelle vidéo est publiée, la chaîne notifie ses abonnés.

```
1 public interface YouTubeChannel {
2     void addSubscriber(Subscriber subscriber);
3     void removeSubscriber(Subscriber subscriber);
4     void notifySubscribers();
5 }
```

```
1 public class YouTubeChannelImpl implements
2     YouTubeChannel {
3     private List<Subscriber> subscribers = new
4     ArrayList<>();
5     private String video;
6
7     @Override
8     public void addSubscriber(Subscriber subscriber)
9     {
10         subscribers.add(subscriber);
11     }
12
13     @Override
14     public void removeSubscriber(Subscriber
15     subscriber) {
16         subscribers.remove(subscriber);
17     }
18
19     @Override
20     public void notifySubscribers() {
21         for (Subscriber subscriber : subscribers) {
22             subscriber.update(video);
23         }
24     }
25
26     public void uploadNewVideo(String video) {
27         this.video = video;
28         notifySubscribers();
29     }
30 }
```

```
1 public interface Subscriber {
2     void update(String video);
3 }
4
5 public class EmailSubscriber implements Subscriber
6 {
7     private String email;
8     public EmailSubscriber(String email) {
9         this.email = email;
10     }
11
12     @Override
13     public void update(String video) {
14         System.out.println("Sending email to " +
15         email + ": New video uploaded: " + video);
16     }
17 }
```

Singleton

Principe

⇒ Permet de s'assurer qu'une classe n'a qu'une seule instance et de fournir un point d'accès global à cette instance.

⇒ Utilisé pour les ressources partagées, les gestionnaires de configuration, etc.

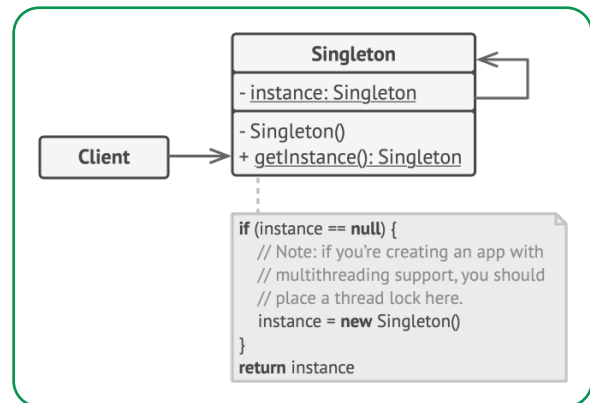


Figure 4: Utilisation simple du pattern Singleton

Exemple

```
1 public final class Singleton {
2     private static Singleton instance;
3     public String value;
4
5     // Constructeur privé
6     private Singleton(String value) {
7         try {
8             Thread.sleep(1000);
9         } catch (InterruptedException ex) {
10             ex.printStackTrace();
11         }
12         this.value = value;
13     }
14
15     // Initialisation à la volée
16     public static Singleton getInstance(String value) {
17         if (instance == null) {
18             instance = new Singleton(value);
19         }
20         return instance;
21     }
22 }
```

```
1 public class DemoSingleThread {
2     public static void main(String[] args) {
3         System.out.println("If you see the same value, then singleton was reused (yay!) " + "\n" +
4             "If you see different values, then 2 singletons were created (boooo!!)" + "\n\n" +
5             "RESULT:" + "\n");
6         Singleton singleton = Singleton.getInstance("FOO");
7         Singleton anotherSingleton = Singleton.getInstance("BAR");
8         System.out.println(singleton.value);
9         System.out.println(anotherSingleton.value);
10     }
11 }
```

⇒ Dans le cas présent, le même singleton est réutilisé dans les deux cas, donc on voit la même valeur "FOO" affichée deux fois.

```
1 If you see the same value, then singleton was reused (yay!)
2 If you see different values, then 2 singletons were created (boooo!!)
3
4 RESULT:
5
6 F00
7 F00
```

Décorateur

Principe

- ⇒ Permet d'ajouter dynamiquement des fonctionnalités à un objet sans modifier sa structure.
- ⇒ Utilisé pour les flux d'entrée/sortie, les interfaces graphiques, etc.
- ⇒ Là où l'héritage est statique, le décorateur permet d'ajouter des fonctionnalités de manière dynamique.
- ⇒ On ne peut hériter que d'une seule classe, mais on peut décorer un objet avec plusieurs décorateurs.

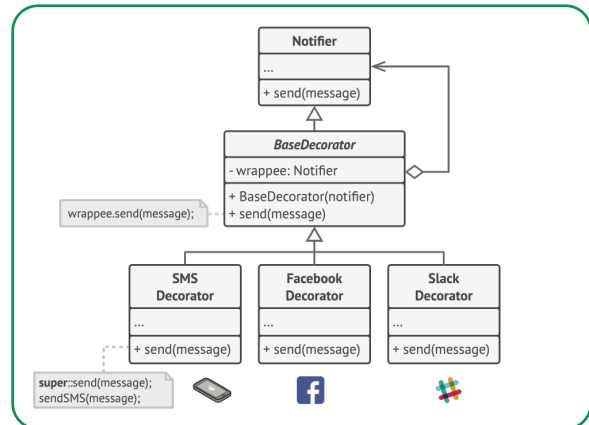


Figure 5: Utilisation simple du pattern Décorateur

Exemple

```
1 // Interface abstraite
2 abstract class DataSource {
3     String fetchData();
4 }
5
6 // Classe concrète
7 class FileDataSource implements DataSource {
8     @Override
9     String fetchData() {
10         return "Hello, World!"; // ⇒ Simulation d'une lecture de fichier
11     }
12 }
13
14 // Décorateur de base
15 class DataSourceDecorator implements DataSource {
16     final DataSource _wrappee;
17     DataSourceDecorator(this._wrappee);
18
19     @Override
20     String fetchData() {
21         return _wrappee.fetchData(); // On ne change rien, la modification est faite dans les décorateurs concrets
22     }
23 }
24
25 // Décorateur 1 : Logueur
26 class LoggingDecorator extends DataSourceDecorator {
27     LoggingDecorator(DataSource wrappee) : super(wrappee);
28
29     @Override
30     String fetchData() {
31         print("[Logging] About to fetch data...");
32         String data = super.fetchData();
33         print("[Logging] Fetched data: $data");
34         return data;
35     }
36 }
37
38 // Décorateur 2 : Transformateur de données
39 class UppercaseDecorator extends DataSourceDecorator {
40     UppercaseDecorator(DataSource wrappee) : super(wrappee);
41
42     @Override-
43     String fetchData() {
44         String data = super.fetchData();
45         return data.toUpperCase(); // met en majuscules
46     }
47 }
```

Usine (Factory)

Principe

⇒ Permet de créer des objets sans exposer la logique de création au client et en utilisant une interface commune.

⇒ Utilisé pour les systèmes de plugins, les bibliothèques de classes, etc.

⇒ Permet de découpler la création d'objets de leur utilisation.

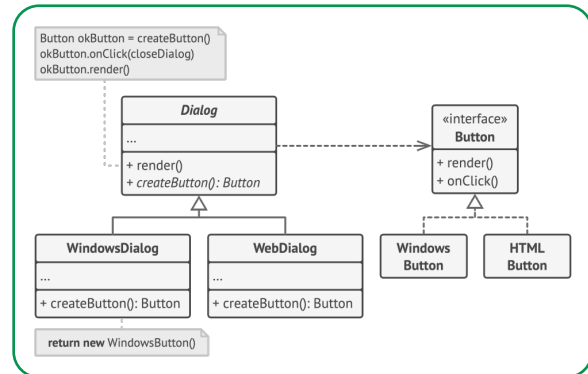


Figure 6: Utilisation simple du pattern Factory

Exemple (Véhicules)

```
1  abstract class Vehicle {
2      public abstract void printWheelNumber();
3  }
4
5  class Moto extends Vehicle {
6      public void printWheelNumber() {
7          System.out.println("2 wheels");
8      }
9  }
10
11 class Car extends Vehicle {
12     public void printWheelNumber() {
13         System.out.println("4 wheels");
14     }
15 }
```

```
1  class VehicleFactory {
2      public static Vehicle createVehicle(String type) {
3          if (type.equalsIgnoreCase("moto")) {
4              return new Moto();
5          } else if (type.equalsIgnoreCase("car")) {
6              return new Car();
7          }
8          throw new IllegalArgumentException("Unknown vehicle type");
9      }
10 }
```

⇒ La factory est généralement un **Singleton** pour éviter d'avoir plusieurs instances de la factory qui pourraient créer des objets de manière incohérente.

⇒ Dans un contexte applicatif, les factories permettent aussi de créer différentes interfaces (CLI, GUI), tout en utilisant les mêmes interfaces pour les objets créés.

Visiteur

Principe

- ⇒ Permet de séparer un algorithme d'une structure d'objets sur laquelle il opère.
- ⇒ Utilisé pour les systèmes de compilation, les systèmes de rendu, etc.
- ⇒ Permet d'éviter les "switch" très gros, de baisser la complexité des algorithmes.

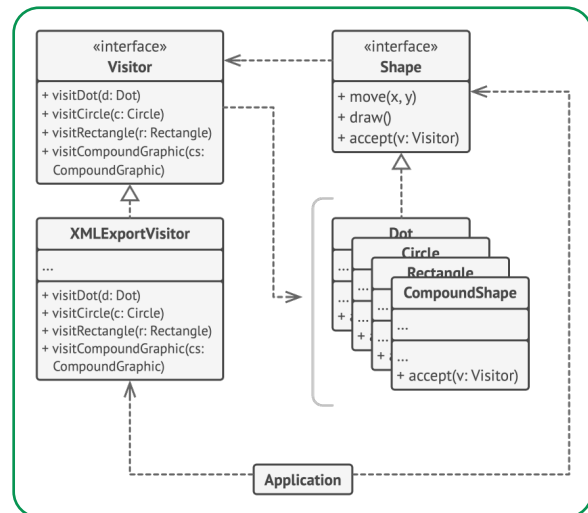


Figure 7: Utilisation simple du pattern Visiteur

Exemple (Debug d'une figure géométrique)

```
1 public interface Shape {
2     // d'autres méthodes pour dessiner la figure, etc.
3     String accept(Visitor visitor);
4 }
```

```
1 public class Circle implements Shape {
2     private int radius;
3     private int id;
4
5     // Constructeur, getters, setters, etc.
6
7     @Override
8     public String accept(Visitor visitor) {
9         return visitor.visitCircle(this);
10    }
11 }
```

```
1 public class Dot implements Shape {
2     private int id;
3
4     // Constructeur, getters, setters, etc.
5
6     @Override
7     public String accept(Visitor visitor) {
8         return visitor.visitDot(this);
9     }
10 }
```

```
1 public interface Visitor {
2     // Une méthode de visite pour chaque type de figure
3     public String visitDot(Dot dot);
4     public String visitCircle(Circle circle);
5 }
```

```
1 public class DebugVisitor implements Visitor {
2     public String visitDot(Dot d) {
3         return "Dot(id=" + d.getId() + ")";
4     }
5
6     public String visitCircle(Circle c) {
7         return "Circle(id=" + c.getId() + ", radius=" + c.getRadius() + ")";
8     }
9 }
```

```
1 public class Main {
2     public static void main(String[] args) {
3         Shape dot = new Dot(1);
4         Shape circle = new Circle(2, 5);
5         Visitor debugVisitor = new DebugVisitor();
6         System.out.println(dot.accept(debugVisitor)); // Affiche les informations du dot
7         System.out.println(circle.accept(debugVisitor)); // Affiche les informations du cercle
8     }
9 }
```


Composite

Principe

- ⇒ Permet de traiter de manière uniforme des objets individuels et des compositions d'objets.
- ⇒ Utilisé pour les systèmes de fichiers, les interfaces graphiques, etc.
- ⇒ Permet de construire des structures d'objets en forme d'arbre.

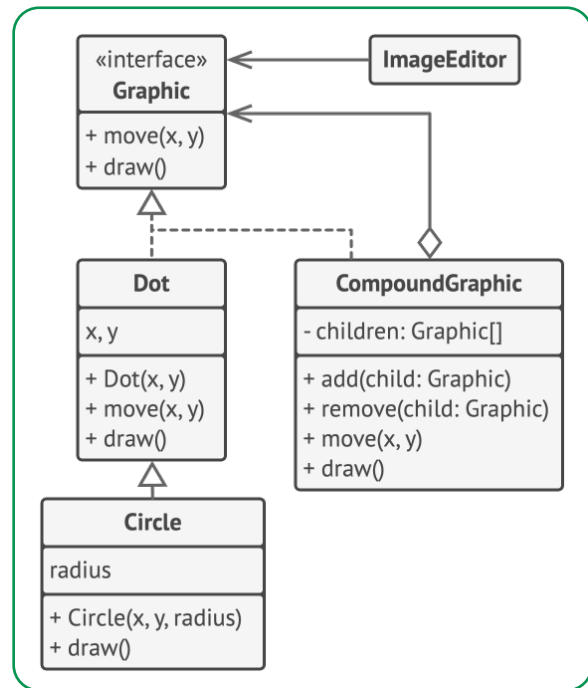


Figure 8: Utilisation simple du pattern Composite

Exemple (Interface graphique)

Dans une interface graphique, on peut avoir une classe `Container` qui contient plusieurs autres éléments graphiques (boutons, champs de texte, rectangles, ...). Le pattern Composite permet de traiter les éléments individuels et les conteneurs de manière uniforme, en utilisant une interface commune.

```
1 public interface Graphic {
2     void draw();
3 }
4
5 public class Button implements Graphic {
6     @Override
7     public void draw() {
8         System.out.println("Drawing a button");
9     }
10 }
11
12 public class TextField implements Graphic {
13     @Override
14     public void draw() {
15         System.out.println("Drawing a text field");
16     }
17 }
```

```
1 public class Container implements Graphic {
2     private List<Graphic> children = new
3     ArrayList<>();
4
5     public void add(Graphic graphic) {
6         children.add(graphic);
7     }
8
9     public void remove(Graphic graphic) {
10        children.remove(graphic);
11    }
12
13    @Override
14    public void draw() {
15        System.out.println("Drawing a container");
16        for (Graphic child : children) {
17            child.draw();
18        }
19    }
20 }
```

Ici, la classe `Container` peut contenir d'autres `Graphic`, qui peuvent être des `Button`, des `TextField`, ou même d'autres `Container`. Lorsqu'on appelle la méthode `draw()` sur un `Container`, elle dessine le conteneur lui-même, puis appelle la méthode `draw()` de tous ses enfants, ce qui permet de dessiner toute la hiérarchie d'éléments graphiques de manière uniforme.

Itérateur

Principe

⇒ Parcours des éléments d'une collection sans exposer sa structure interne.

⇒ Utilisé pour les collections, les arbres, etc.

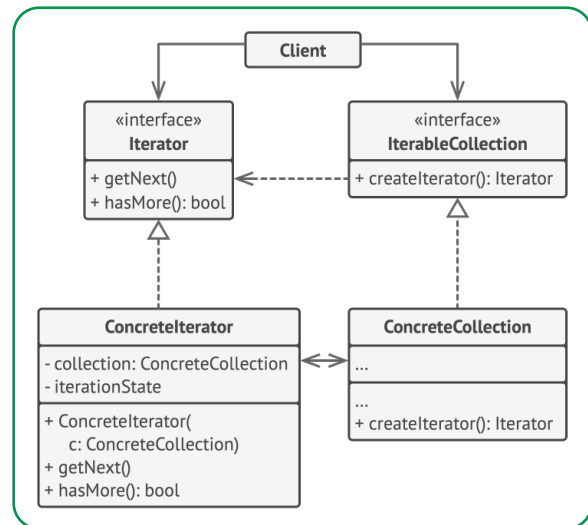


Figure 9: Principe du design pattern Itérateur

Exemple (Bibliothèque)

```
1 public record Book(String title, String author) {}
2
3 public class Library implements BookCollection {
4     private List<Book> books = new ArrayList<>();
5
6     @Override
7     public void addBook(Book book) {
8         books.add(book);
9     }
10
11     @Override
12     public Iterator<Book> createIterator() {
13         return new BookIterator(books);
14     }
15 }
```

```
1 public class Main {
2     public static void main(String[] args) {
3         Library library = new Library();
4         library.addBook(new Book("The Great Gatsby",
5             "F. Scott Fitzgerald"));
6         library.addBook(new Book("To Kill a
7             Mockingbird", "Harper Lee"));
8
9         Iterator<Book> iterator =
10         library.createIterator();
11         while (iterator.hasNext()) {
12             Book book = iterator.next();
13             System.out.println(book.getTitle() + "
14             by " + book.getAuthor());
15         }
16     }
17 }
```

Façade

Template